

Passo a Passo de Implementação:

Passo 1: Estruturar o HTML e Conectar o CSS Externa

1. Crie o arquivo index.html e defina a estrutura básica do documento com a tag <title>Validação de Formulário com Máscara de CPF - DOM</title>.
2. Dentro da tag <head>, conecte o arquivo de estilos externo utilizando a tag: <link rel="stylesheet" href="style.css">.
3. No <body>, crie um título <h2>Cadastro de Usuário</h2> e organize os formulários dentro de contêineres <div class="campo-container">.
4. Adicione os campos de entrada com seus respectivos identificadores:
 - o **Nome:** <input type="text" id="campoNome">
 - o **CPF:** <input type="text" id="campoCpf" maxlength="14">
 - o **E-mail:** <input type="email" id="campoEmail">
 - o **Data de Nascimento:** <input type="date" id="campoData">
 - o **Senha:** <input type="password" id="campoSenha">
5. Adicione o botão <button id="btnCadastrar">Cadastrar</button> e a caixa de feedback <div id="painelResultado"></div>.

Passo 2: Mapear os Elementos com JavaScript

1. Abra a tag <script> no final do arquivo HTML.
2. Mapeie cada elemento da tela criando constantes com document.getElementById().
3. Agrupe todos os campos em uma lista array const listaCampos = [campoNome, campoCpf, campoEmail, campoData, campoSenha]; para facilitar a validação em lote.

Passo 3: Implementar a Máscara Dinâmica no Campo de CPF

1. Vincule o escutador de eventos 'input' ao elemento campoCpf.
2. Utilize expressões regulares (replace) para remover qualquer caractere não numérico (\D).
3. Formate a string em tempo real aplicando a pontuação padrão do CPF (000.000.000-00) enquanto o usuário digita.

```
4. // 2. Máscara Dinâmica e Restrição de Apenas Números no CPF
5.     campoCpf.addEventListener('input', function() {
6.         let valor = campoCpf.value.replace(/\D/g, "");
7.
8.         valor = valor.replace(/(\d{3})(\d)/, '$1.$2');
9.         valor = valor.replace(/(\d{3})(\d)/, '$1.$2');
10.        valor = valor.replace(/(\d{3})(\d{1,2})$/, '$1-$2');
11.
12.        campoCpf.value = valor;
```

```
13.     });
```

Passo 4: Criar a Função para Gerar e Baixar o Arquivo TXT

1. Crie a função `salvarDadosEmTXT(dados)` que aceita o objeto de dados do usuário.
2. Formate o texto final utilizando Template Strings para organizar os dados em linhas claras.
3. Instancie um objeto `new Blob()` do tipo `'text/plain'` na memória do navegador.
4. Crie dinamicamente um elemento `<a>` invisível no DOM, defina o atributo `download` para nomear o arquivo (ex: `cadastro_nome.txt`) e simule o evento `.click()` automático para disparar o download.

```
5. // 3. Função para Gerar e Baixar o Arquivo TXT no Navegador
6.     function salvarDadosEmTXT(dados) {
7.         // Formata o texto que vai pra dentro do arquivo TXT
8.         const conteudoTXT = `=== DADOS DO CADASTRO ===\n` +
9.             `Nome: ${dados.nome}\n` +
10.            `CPF: ${dados.cpf}\n` +
11.            `E-mail: ${dados.email}\n` +
12.            `Data de Nascimento: ${dados.dataNascimento}\n` +
13.            `Senha: ${dados.senha}\n` +
14.            `===== \n`;
15.
16.            // Cria um Blob (Binary Large Object) simulando o
            // arquivo de texto na memória do navegador
17.            const blob = new Blob([conteudoTXT], { type:
18.                'text/plain;charset=utf-8' });
19.
20.            // Cria uma tag <a> oculta para disparar o download
            // automaticamente
21.            const linkDownload = document.createElement('a');
22.            linkDownload.href = URL.createObjectURL(blob);
23.            linkDownload.download =
24.                `cadastro_${dados.nome.toLowerCase().replace(/\s+/g, '_')}.txt`;
25.
26.            // Simula o clique do usuário para iniciar o download e
            // limpa a URL criada
27.            document.body.appendChild(linkDownload);
28.            linkDownload.click();
29.            document.body.removeChild(linkDownload);
```

```
28.         URL.revokeObjectURL(linkDownload.href);
29.     }
```

Passo 5: Validar os Campos e Executar o Redirecionamento

1. Adicione o evento de clique ('click') no botão btnCadastrar.
2. Percorra a listaCampos utilizando .forEach() e verifique se algum campo está vazio usando .trim() === ''.
3. Se houver campo vazio, aplique a classe CSS .campo-erro ao input e exiba a mensagem de alerta no painelResultado.
4. Se todos os campos estiverem preenchidos:
 - Remova as marcações de erro.
 - Organize as informações digitadas em um objeto dadosUsuario.
 - Chame a função salvarDadosEmTXT(dadosUsuario) para exportar o arquivo.
 - Exiba a mensagem de sucesso e utilize a função setTimeout() para aguardar 2 segundos (2000ms) antes de redirecionar para a tela de login via window.location.href = 'login.html';.

```
5. btnCadastrar.addEventListener('click', function() {
6.     let temCampoVazio = false;
7.
8.     listaCampos.forEach(campo => {
9.         if (campo.value.trim() === '') {
10.            campo.classList.add('campo-erro');
11.            temCampoVazio = true;
12.        } else {
13.            campo.classList.remove('campo-erro');
14.        }
15.    });
16.
17.    if (temCampoVazio) {
18.        divResultado.className = 'msg-erro';
19.        divResultado.innerText = 'Atenção: Preencha todos
os campos destacados!';
20.        return;
21.    }
22.
23.    // Organiza os dados capturados em um objeto
24.    const dadosUsuario = {
25.        nome: campoNome.value.trim(),
26.        cpf: campoCpf.value.trim(),
```

```
27.         email: campoEmail.value.trim(),
28.         dataNascimento: campoData.value,
29.         senha: campoSenha.value.trim()
30.     };
31.
32.     // Dispara o salvamento e o download do arquivo .txt
33.     salvarDadosEmTXT(dadosUsuario);
34.
35.     // Mensagem de confirmação na tela
36.     divResultado.className = 'msg-sucesso';
37.     divResultado.innerText = `Cadastro de
    ${dadosUsuario.nome} realizado com sucesso! Redirecionando para a
    tela de login...`;
38.
39.     // Redireciona para o login após 2 segundos
40.     setTimeout(function() {
41.         window.location.href = 'login.html';
42.     }, 2000);
43. });
```